

Computer Organization and Architecture Laboratory

Practice Assignment: Using BRAM as ROM and RAM

Indian Institute of Technology Kharagpur
Department of Computer Science and Engineering

Objective

This is a **practice assignment** intended to familiarize students with FPGA Block RAM (BRAM) in both ROM and RAM configurations.

The complete design is to be verified using **behavioral simulation only**. An FPGA-board demonstration is not required.

Students will design a controller that:

1. reads a sequence of addresses from BRAM configured as ROM;
2. uses each ROM value as an address for a second BRAM configured as RAM;
3. reads the corresponding 32-bit RAM value;
4. accumulates all eight values;
5. writes the final sum back to RAM address 15;
6. reads address 15 again and presents the stored value as the final output.

The assignment provides practice with BRAM IP configuration, .coe initialization, synchronous memory timing, RAM read/write control, and simple FSM sequencing.

1. Required Top-Level Interface

Create:

```
module bram_rom_ram_practice (  
    input wire      clk,  
    input wire      reset,  
    input wire      start,  
    output reg      done,  
    output reg [31:0] result_out  
);
```

Signal	Width	Description
clk	1	System clock
reset	1	Active-high controller reset
start	1	Starts one complete operation
done	1	Asserted after final RAM read-back
result_out	32	Value read back from RAM address 15

After reset, the circuit must wait in IDLE until **start** is asserted. One start pulse must cause the entire eight-entry operation to execute automatically.

2. BRAM 1: Address ROM

Create a BRAM IP named `address_rom` configured as:

- Single-Port ROM;
- depth = 8 words;
- width = 4 bits;
- address width = 3 bits;
- synchronous read with one-clock read latency;
- no additional optional output pipeline register.

Conceptual interface:

```
address_rom rom_inst (  
    .clka   (clk),  
    .ena    (rom_en),  
    .addra  (rom_addr),    // 3 bits  
    .douta  (rom_data)     // 4 bits  
);
```

Initialize the ROM as:

ROM Address	Stored Value
0	2
1	5
2	1
3	7
4	3
5	0
6	6
7	4

Thus the RAM-address sequence is

`2, 5, 1, 7, 3, 0, 6, 4`.

Suggested `address_rom.coe`:

```
memory_initialization_radix=16;  
memory_initialization_vector=  
2,  
5,  
1,  
7,  
3,  
0,  
6,  
4;
```

3. BRAM 2: Data RAM

Create a BRAM IP named `data_ram` configured as:

- Single-Port RAM;
- depth = 16 words;
- width = 32 bits;
- address width = 4 bits;
- synchronous read with one-clock read latency;
- one write-enable control;
- no additional optional output pipeline register.

Conceptual interface:

```
data_ram ram_inst (  
    .clka  (clk),  
    .ena   (ram_en),  
    .wea   (ram_we),  
    .addra (ram_addr),      // 4 bits  
    .dina  (ram_write_data), // 32 bits  
    .douta (ram_read_data)  // 32 bits  
);
```

Initialize:

RAM Address	Initial Value
0	10
1	20
2	30
3	40
4	50
5	60
6	70
7	80
8–15	0

Suggested `data_ram.coe`:

```
memory_initialization_radix=16;  
memory_initialization_vector=  
0000000A,  
00000014,  
0000001E,  
00000028,  
00000032,  
0000003C,  
00000046,  
00000050,  
00000000,  
00000000,
```

```

00000000,
00000000,
00000000,
00000000,
00000000,
00000000;

```

4. Required Computation

For ROM index $i = 0, 1, \dots, 7$:

$$ACC \leftarrow ACC + RAM[ROM[i]].$$

The expected sequence is:

ROM Index	RAM Address	RAM Value	ACC After Addition
0	2	30	30
1	5	60	90
2	1	20	110
3	7	80	190
4	3	40	230
5	0	10	240
6	6	70	310
7	4	50	360

Hence,

$$ACC = 360 = 32'h00000168.$$

5. Final RAM Write and Read-Back

After the eighth accumulation:

$$RAM[15] \leftarrow 32'h00000168.$$

Then perform a normal read of RAM address 15 and copy that value into `result_out`.

Only after the read-back completes should

$$\text{done} = 1$$

be asserted.

Expected final output:

$$\text{result_out} = 32'h00000168$$

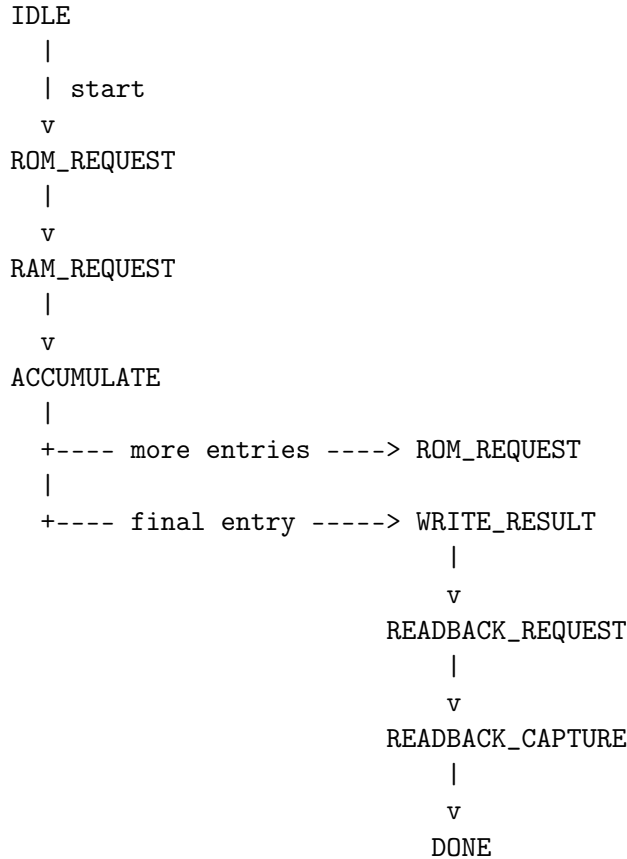
and

done = 1.

Important: `result_out` must come from reading RAM address 15 back after the write. It must not simply be connected to the accumulator.

6. Required Controller Behavior

A small FSM must sequence the memory operations. A suitable state sequence is:



Equivalent FSM organizations are allowed if BRAM latency is handled correctly.

State meanings:

- **IDLE:** wait for `start`;
- **ROM_REQUEST:** apply current ROM index;
- **RAM_REQUEST:** use ROM output as RAM address;
- **ACCUMULATE:** add valid RAM output to accumulator;
- **WRITE_RESULT:** write final accumulator value to RAM[15];
- **READBACK_REQUEST:** disable write and request RAM[15];
- **READBACK_CAPTURE:** capture RAM[15] into `result_out`;
- **DONE:** assert `done` and hold the result.

7. Synchronous BRAM Timing

Students must not treat the BRAMs as combinational memories.

For example:

Clock n : $ROM\ address = i$

and only in the following cycle may

$ROM[i]$

be used as valid ROM output.

Likewise:

Clock m : $RAM\ address = ROM[i]$

and only in the following cycle may

$RAM[ROM[i]]$

be accumulated.

Students must verify the exact timing using the generated BRAM behavioral model.

8. Reset and Start Behavior

On **reset**:

- clear the accumulator;
- clear the ROM index;
- clear **done**;
- clear **result_out**;
- return the FSM to IDLE.

Reset is intended to reset the controller. It does not need to rewrite the contents initialized inside the BRAM IPs.

After reset is released, assert **start** for at least one active clock edge.

Only one complete operation after each reset is required.

9. Required Behavioral Testbench

Write a behavioral testbench that:

1. generates a clock;

2. applies reset;
3. releases reset;
4. pulses **start**;
5. waits until **done**=1;
6. checks
7. prints PASS or FAIL;
8. terminates using **\$finish**.

`result_out = 32'h00000168;`

A suitable checking structure is:

```
wait(done == 1'b1);

if (result_out == 32'h00000168)
    $display("PASS");
else
    $display("FAIL: result = %h", result_out);

$finish;
```

10. Signals to Show in the Waveform

Show at least:

- `clk`;
- `reset`;
- `start`;
- FSM state;
- ROM index/address;
- ROM output;
- RAM address;
- RAM read data;
- RAM write data;
- RAM write enable;
- accumulator;
- `result_out`;
- `done`.

The waveform must make the sequence

ROM Address → ROM Output → RAM Address → RAM Data → Accumulator

clearly visible.

11. Mandatory Restrictions

1. Both memories must use FPGA BRAM IP.
2. The first memory must be configured as ROM.
3. The second memory must be configured as RAM.
4. Neither memory may be replaced by a Verilog register array.
5. The supplied memory contents must be loaded using initialization files.
6. Synchronous BRAM read latency must be handled correctly.
7. The final sum must be written to RAM address 15.
8. `result_out` must be obtained by reading address 15 back.
9. The value 360 must not be hard-coded as the output.

12. Suggested Module Organization

```
bram_rom_ram_practice
|
|-- address_rom          // BRAM IP: 8 x 4 ROM
|-- data_ram             // BRAM IP: 16 x 32 RAM
|-- control_fsm
|-- rom_index_counter
'-- accumulator_32
```

Students may organize the controller differently, but the two BRAM IPs must remain clearly identifiable.

13. Expected Final Result

For the supplied initialization values:

$$30 + 60 + 20 + 80 + 40 + 10 + 70 + 50 = 360.$$

Therefore,

$$RAM[15] = 32'h00000168$$

and after the mandatory read-back:

$$result_out = 32'h00000168, \quad done = 1.$$

14. Practice Verification

Students should show:

- top-level Verilog;
- controller/FSM Verilog;
- generated BRAM IP configuration;
- `address_rom.coe`;
- `data_ram.coe`;
- behavioral testbench;
- behavioral simulation waveform;

No synthesis, implementation, timing simulation, or FPGA-board demonstration is required.